HealMesh — Technical Design Document (TDD)

Status: Draft v1 **Relationship to current repo:** Replaces the n8n workflow + OpenClaw HTTP integration described in the existing README. Retains and renames the Kubernetes namespace, Helm chart structure, and RBAC approach (`prism` → `healmesh`) where they don't conflict with the security invariants below. The incident JSON contract shape from `backend/CONTRACTS.md` is a reasonable starting point and should be preserved/adapted rather than redesigned from zero.

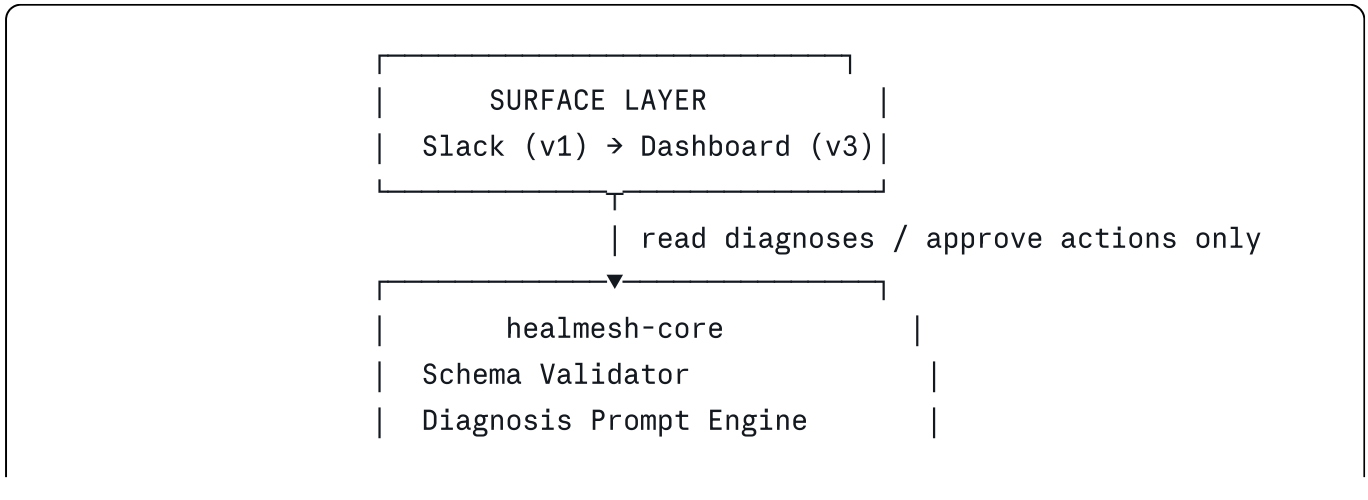
---

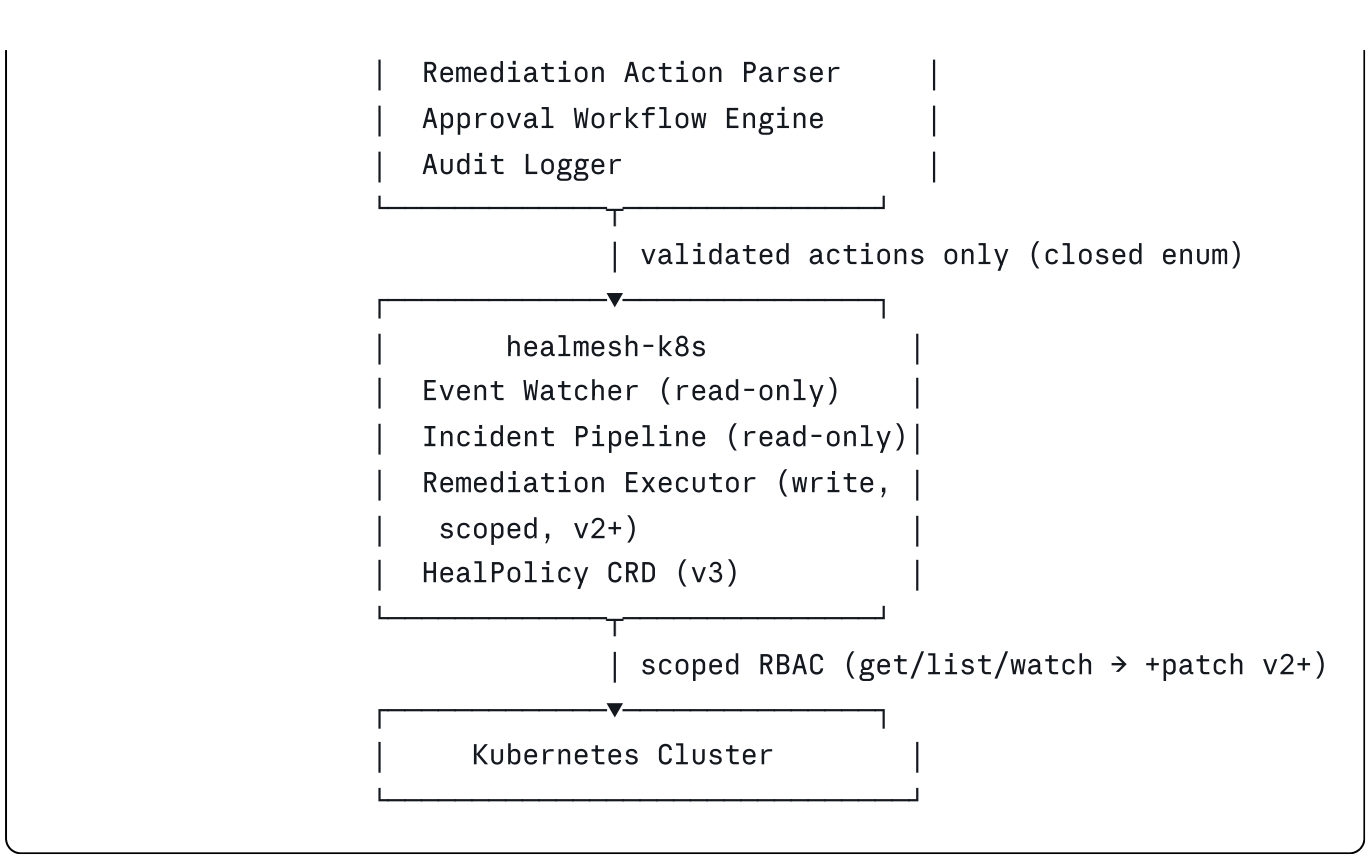
1. Design Principles (non-negotiable, apply to every phase)

1. **The LLM never produces an executed command.** Its output is always parsed into a closed, typed enum (`PATCH | REDEPLOY | SCALE | HELM_UPGRADE | NONE`) with strictly typed parameters. Unparseable output → `NONE`, diagnosis-only fallback. This is the single load-bearing security decision in the system.
2. **No orchestration middleman between the core logic and the LLM API.** Direct API calls only. No n8n, no third-party agent gateway. This keeps the security-critical path — schema validation → prompt → parse → enum — inside code the team fully owns and can audit line by line.
3. **Read and write are architecturally separate.** The component that watches the cluster (read) is never the same component that can act on it (write). Even within "the operator," these are separate code paths with separate RBAC scopes.
4. **Secure by default, opt-in to widen.** Every new namespace, every new action type, every new integration starts with the narrowest possible permission and is widened explicitly, never the reverse.
5. **Audit trail is append-only at the code level, not just the database level.** No delete/update method exists in the code for audit records — defense in depth beyond DB permissions/triggers.

---

2. System Architecture





3. Component Design

3.1 `healmesh-k8s` — Event Watcher (Phase 1)

- Language: Python (`kubernetes` client library) or Go (`client-go`) — pick based on team's stronger language; Python is faster to iterate on for Phase 1, Go is the more idiomatic long-term choice for an operator. **Decision needed before Phase 1 kickoff** (see Implementation Plan).
- Subscribes to Pod, Deployment, and Event resources via the Kubernetes watch API. No polling loops.
- Filters events down to the five canonical failure signatures (see 3.4 below for detection logic per type).
- On detection, assembles an incident payload: pod name, namespace, failure type, last N log lines (bounded, see 3.6), relevant deployment spec fields, timestamp.
- Sends the incident payload to `healmesh-core` via internal HTTP call (mTLS or at minimum TLS within cluster).
- **RBAC (Phase 1):** `get`, `list`, `watch` on `pods`, `deployments`, `events` in the target namespace only. Nothing else. No `patch`, `update`, `delete`, `create`, `exec`.

3.2 `healmesh-core` — Diagnosis Engine

Schema Validator

- JSON Schema (or Pydantic model) defining the exact required shape of an incoming incident.
- Reject and log (not silently drop) anything that doesn't validate. This is the first security gate.

Diagnosis Prompt Engine

- Builds a structured prompt from the validated incident. Prompt design is failure-type-specific but shares a common template (see 3.5).
- Calls the LLM provider directly via its API (Anthropic or OpenAI SDK — see 3.7 for selection criteria). No intermediate gateway.
- Requests a structured output (JSON mode / tool-use / function-calling, whichever the chosen provider supports most reliably) so the response is machine-parseable, not just prose.

Remediation Action Parser

- Takes the LLM's structured response and validates it against the closed enum + typed parameters.
- Any deviation — extra fields, wrong types, an action not in the enum, missing required parameters — results in the action being set to `NONE` and the diagnosis being delivered without a suggested automated action (a suggested manual command as plain text is still fine — that's just text for a human to read, not something the parser needs to trust).
- This component should be small, pure, and have full unit test coverage — it is the most security-critical function in the codebase.

Approval Workflow Engine (Phase 2+)

- Given a parsed action and its risk tier, routes to either auto-eligible (none, initially — even `SCALE` requires approval at launch of Phase 2 per PRD) or approval-required.
- Approval-required actions generate a Slack message with an approve/reject interaction, tied to the incident ID.
- Records who approved/rejected and when.

Audit Logger

- Writes every incident, diagnosis, parsed action, approval decision, and (once it exists) execution result to Postgres.
- No `UPDATE`/`DELETE` methods exist in this component's interface. Combined with a DB-level trigger blocking those operations on the table (belt and suspenders).

3.3 `healmesh-k8s` — Remediation Executor (Phase 2+)

- Separate code path and separate service account identity from the Event Watcher, even if co-located in the same pod for simplicity early on.
- Accepts only the closed-enum, pre-approved action type from `healmesh-core` — the function signature itself should make it impossible to pass a raw string or shell command.

- Calls the Kubernetes API directly for the action (e.g., `patch` on a Deployment's `replicas` field for `SCALE`). No shell-outs, no `subprocess` calls to `kubectl`.
- **Hardcoded namespace denylist** (`kube-system`, `kube-public`, `healmesh`'s own namespace) checked in code before any write call — not configurable, not overridable by `HealPolicy`.
- Takes a pre-action snapshot of the resource being modified (store the prior spec) before writing.
- **RBAC (Phase 2, `SCALE` only):** adds `patch` on the `scale` subresource of Deployments in namespaces explicitly enabled via `HealPolicy` (or a simple config list before the CRD exists). Nothing broader.

3.4 Failure Detection Logic (per canonical type)

Failure type	Detection signal
`CrashLoopBackOff`	Pod status reason field; restart count crossing threshold within time window
`OOMKilled`	Container `lastState.terminated.reason == OOMKilled`
`ImagePullBackOff`	Pod status reason field on container status
Failed rollout	Deployment status conditions showing `ProgressDeadlineExceeded` or replica mismatch beyond timeout
Resource quota exceeded	Event reason `FailedCreate`/`FailedScheduling` with quota-related message pattern

3.5 Prompt Contract (shared template, filled per failure type)

Inputs: `failure_type`, `pod_name`, `namespace`, `restart_count` (if applicable), `last_50_log_lines` (truncated/sanitized), `relevant_spec_fields` (resource limits, image, probes — only fields relevant to the failure type, not the full spec).

Required output shape: `{ root_cause: string, confidence: "high"|"medium"|"low", suggested_action: { type: enum, params: {...} } | null, suggested_manual_command: string }`

The `suggested_manual_command` field is always plain text for the human to read and run themselves — it is never fed back into the executor, even after Phase 2. Only `suggested_action` (the typed enum) is ever eligible for execution.

3.6 Input Sanitization

- Log content is truncated to a fixed line/character budget before entering the prompt.

- Basic sanitization pass to strip patterns resembling secrets/credentials (API keys, tokens, connection strings) from log content before it's sent to the LLM or stored — defense against both accidental leakage and prompt-injection-via-log-content from a compromised container.

3.7 LLM Provider Selection

- Evaluate Claude and GPT-4o-class models during Phase 0 on: structured-output reliability, latency, cost per incident, and accuracy on the benchmark set. Document the decision and rationale once made — don't leave this implicit.
- Use the provider's native structured-output feature (tool-use/function-calling or JSON mode) rather than asking for JSON in free text and hoping the format holds.

4. Data Layer

- **Postgres:** `incidents`, `diagnoses`, `actions`, `approvals` tables. Append-only trigger on all of them, not just the audit-specific table.
- **Secrets:** Kubernetes Secrets are acceptable for Phase 1 given single-cluster scope; migrate to Vault (or equivalent) no later than Phase 2, before any write-capable credential exists (the `SCALE` execution service account token is exactly the kind of credential that should never sit in a plain Secret longer than necessary).

5. Networking & Runtime Security

- TLS between all internal services (cert-manager for internal CA, even in single-cluster Phase 1 — cheap to set up early, expensive to retrofit).
- NetworkPolicy restricting each pod to exactly the other services it needs (Event Watcher → healmesh-core only; healmesh-core → LLM API + Postgres + Slack only).
- Slack webhook handler verifies HMAC signature as the first line of request handling, before any payload parsing.
- Container images: minimal base images, non-root user, dependency scanning in CI before any deploy.

6. What Carries Over From the Current Repo

Current artifact	Disposition
`infra/helm/prism/`	Rename to `infra/helm/healmesh/`; restructure from multi-service (n8n, OpenClaw, Postgres, Redis) chart to a leaner chart (healmesh-core, healmesh-k8s,

Current artifact	Disposition
	Postgres) as components are rebuilt
Kubernetes namespace `prism`	Rename to `healmesh`
`infra/scripts/remediate.sh`	Superseded by the in-process Remediation Executor (3.3) — the enum-to-action mapping this script encodes is useful reference for defining the typed action parameters, but execution should happen via the Kubernetes API client, not a shell script
`backend/CONTRACTS.md`	Adapt into the formal incident/diagnosis JSON schema for `healmesh-core` — the payload shape is a reasonable starting point
`backend/workflows/prism-incident.json` (n8n)	Retired. Logic re-implemented as `healmesh-core`
OpenClaw integration	Retired. Replaced by direct LLM API calls from `healmesh-core`
`backend/migrations/001_remediation_incidents.sql`	Review and adapt as the starting point for the Postgres schema (3.4 in this doc) rather than writing from scratch

7. Explicitly Deferred (documented, not forgotten)

- `HealPolicy` CRD — designed conceptually now (per-namespace allowed actions), implemented in Phase 3 once there's an actual policy decision worth configuring.
- Dashboard — Slack is sufficient until incident volume makes a channel unwieldy.
- SDK — valuable once the core diagnosis is proven; premature before that.
- Blockchain audit anchoring — protects an audit trail that doesn't have meaningful volume yet; revisit once there's a real customer with compliance requirements driving it.
- Vault — acceptable to defer to Phase 2 given Phase 1 has no write-capable credentials, but must land before any execution capability ships.